Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 07

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad avanzada del ORM de Django (transacciones, expresiones F, agregaciones y QuerySets personalizados) dentro de una aplicación empresarial.**
2. **Registrar operaciones de forma segura con transaction.atomic() y F(), y construir reportes con aggregate() y annotate() sobre modelos relacionados.**
3. **Encapsular consultas de negocio en un QuerySet personalizado y optimizar su rendimiento con select_related y prefetch_related, midiendo el número de consultas.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 7 (ORM avanzado: transacciones y F(), aggregate() y annotate(), QuerySet personalizado y problema N+1) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 (CRUD con URLs, Views y Templates), ampliada con relaciones en la Semana 4, registrada en el Django Admin en la Semana 5 y con Templates que heredan de base.html desde la Semana 6, y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (models.py, views.py y Template correspondiente) y del resultado en el navegador o en el shell de Django, incluyendo el número de consultas medido antes y después de optimizar.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con URLs, Views y Templates propios que implementan CRUD completo, ampliados en la Semana 4 con relaciones, registrados en el Django Admin en la Semana 5 y con Templates que heredan de base.html desde la Semana 6. En esta sesión no se agregan entidades nuevas: se aprovechan las que ya existen para registrar operaciones de forma segura y generar reportes con consultas avanzadas del ORM. A diferencia

de la Semana 6, sí se crean una View y un Template nuevos para mostrar el reporte; el Template debe heredar de base.html.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de las Semanas 4 a 6 y aplica transacciones, agregaciones, un QuerySet personalizado y la optimización de consultas. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que las consultas avanzadas no reemplazan al ORM que ya conoce: lo aprovechan. Las mismas relaciones de la Semana 4 permiten calcular totales, agrupar resultados, encapsular reglas de negocio y reducir el número de consultas a la base de datos. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	**Equivalente en su aplicación**
Campo entero que se descuenta (en clase: stock de Producto)	Un campo numérico entero de su aplicación (existencias, cupos, unidades disponibles, copias).
Campo de estado (en clase: estado de Pedido)	Un campo de texto con valores como PENDIENTE, PAGADO, ACTIVO o DEVUELTO.
Atributos del modelo intermedio (en clase: cantidad y precio_unitario de DetallePedido)	Los atributos propios de su modelo intermedio N:M (cantidad, monto, multa, horas, fecha).
Operación que modifica varios modelos (en clase: registrar un pedido)	Una operación de su aplicación que crea o modifica más de un registro a la vez.

Cliente (navegador)

→ URL nueva (ej. /reporte/) en urls.py

→ View nueva (views.py)

→ QuerySet personalizado (Manager)

→ Django ORM (aggregate / annotate)

→ SQLite

→ Template nuevo (extends base.html)

→ Response

Al finalizar el laboratorio, el estudiante deberá ser capaz de registrar una operación con transaction.atomic() y F(), construir reportes con aggregate() y annotate(), encapsular consultas en un QuerySet personalizado con as_manager(), y medir y reducir el número de consultas con select_related y prefetch_related.

Enunciados

PARTE 1 — Consultas avanzadas sobre la aplicación de las Semanas 4 a 6

Ejercicio 1 — Recuperar la aplicación y ubicar sus campos clave

Abra la aplicación desarrollada hasta la Semana 6 e identifique su entidad principal y sus tres relaciones (OneToOneField, ForeignKey y ManyToManyField con through). Con ayuda de la tabla de equivalencias, ubique tres campos: un campo entero que pueda descontarse, un campo de estado y un atributo numérico del modelo intermedio. Si alguno no existe, agréguelo al Model, ejecute makemigrations y migrate, y documente el cambio con una captura del archivo de migración generado.

```python
class Servicio(models.Model):
    nombre = models.CharField(max_length=100)
    duracion_min = models.IntegerField()
    precio = models.DecimalField(max_digits=8, decimal_places=2)
    # 1. Campo entero que se descuenta (Equivalente al stock del lab
    cupos_disponibles = models.PositiveIntegerField(default=50)

    def __str__(self):
        return self.nombre
```

```python
class PacienteServicio(models.Model):
    paciente = models.ForeignKey('Paciente', on_delete=models.CASCADE)
    servicio = models.ForeignKey(Servicio, on_delete=models.CASCADE)

    # 2. Atributos numéricos del modelo intermedio
    cantidad = models.PositiveIntegerField(default=1)
    monto = models.DecimalField(max_digits=8, decimal_places=2, default=0.00)

    # 3. Campo de estado
    ESTADOS = [
        ('PENDIENTE', 'Pendiente'),
        ('PAGADO', 'Pagado'),
        ('DEVUELTO', 'Devuelto')
    ]
    estado = models.CharField(max_length=20, choices=ESTADOS, default='PENDIENTE')
    fecha_realizacion = models.DateField(auto_now_add=True)

    def __str__(self):
        return f"{self.paciente.nombre} - {self.servicio.nombre} ({self.estado})"
```

```
Problems  Output  Debug Console  Terminal  Ports                    powershell - Semana_07

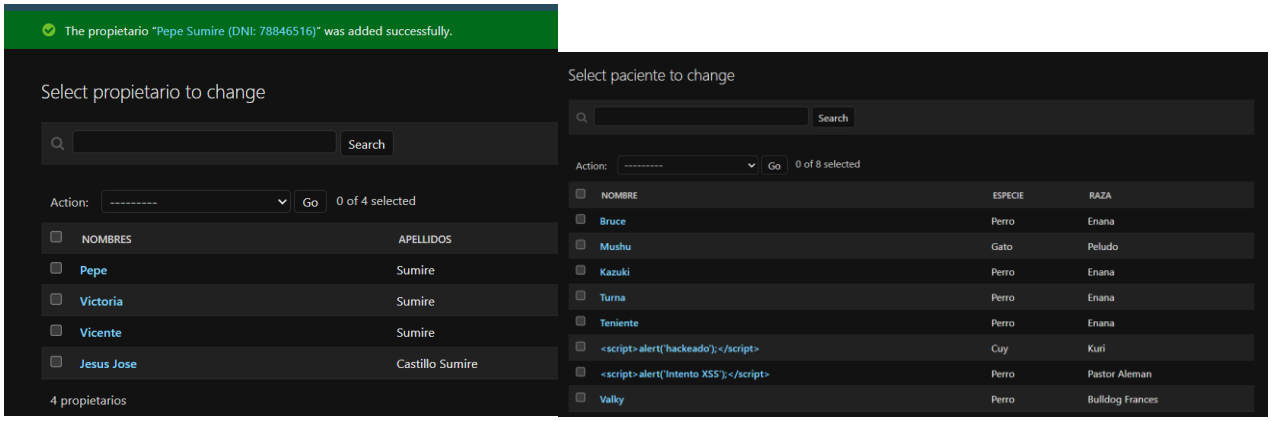
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_07> python manage.py makemigrations gestion_clinica
>> python manage.py migrate
    ~ Alter field estado on pacienteservicio
    ~ Alter field precio on servicio
C:\Users\BossC\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\django\db\models\base.py:368: RuntimeWarning: Model 'gestion_clinica.pacienteservicio' was already registered. Reloading models is not advised as it can lead to inconsistencies, most notably with related models.
  new_class._meta.apps.register_model(new_class._meta.app_label, new_class)
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion_clinica, sessions
Running migrations:
  Applying gestion_clinica.0005_rename_duracion_minutos_servicio_duracion_min_and_more... OK
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_07> python manage.py migrate
C:\Users\BossC\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\django\db\models\base.py:368: RuntimeWarning: Model 'gestion_clinica.pacienteservicio' was already registered. Reloading models is not advised as it can lead to inconsistencies, most notably with related models.
  new_class._meta.apps.register_model(new_class._meta.app_label, new_class)
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion_clinica, sessions
Running migrations:
  No migrations to apply.
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_07>
```

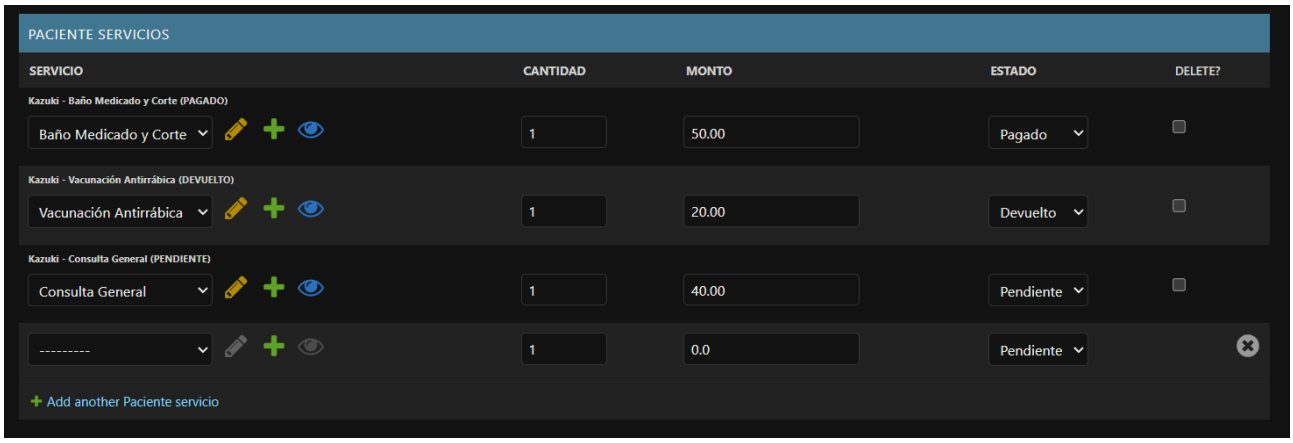
Recuperé la aplicación de la Semana 6 copiándola en un nuevo directorio aislado (Semana 07). Identifiqué que mi entidad principal es Paciente y que la relación elegida para las consultas avanzadas será la N:M con Servicio a través del modelo intermedio PacienteServicio. Basándome en la tabla de equivalencias de la guía, actualicé mi archivo models.py de la siguiente manera:

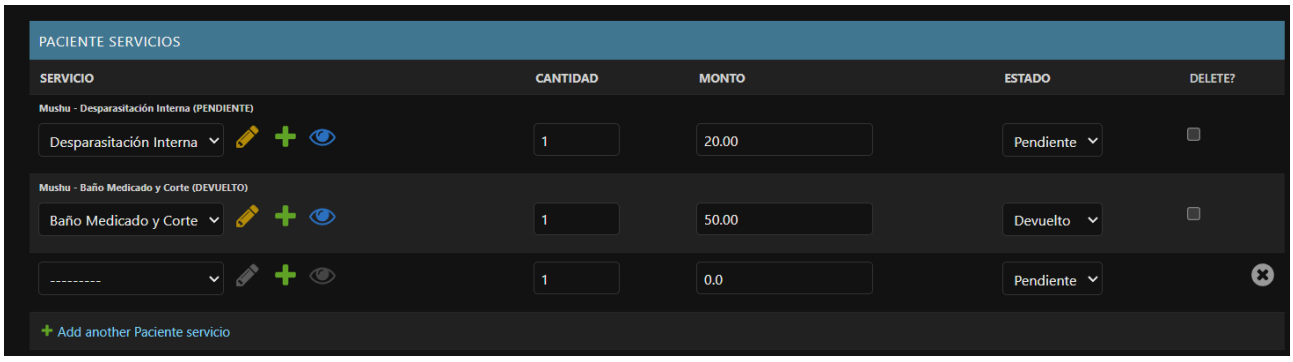
1. **Campo entero a descontar: Agregué el campo cupos_disponibles a la entidad Servicio.**
2. **Campo de estado: Agregué el campo estado (PENDIENTE, PAGADO, DEVUELTO) a PacienteServicio.**
3. **Atributos numéricos del modelo intermedio: Añadí cantidad y monto a PacienteServicio. Finalmente, ejecuté makemigrations y migrate para reflejar estos cambios estructurales en la base de datos."**

Ejercicio 2 — Cargar datos de prueba

Desde el Django Admin (Semana 5), registre datos suficientes para que los reportes tengan resultados: al menos cinco registros de la entidad principal, tres de la entidad de la relación 1:N y ocho del modelo intermedio N:M, con estados y cantidades variadas. Documente con una captura el listado en el Admin.







Para asegurar que los reportes posteriores tuvieran datos suficientes para procesar, ingresé al panel de administración de Django (implementado en la Semana 5) y poblé la base de datos. Específicamente, registré tres dueños en la entidad de la relación 1:N (Propietario), cinco mascotas en la entidad principal (Paciente), y ocho transacciones en el modelo intermedio N:M (PacienteServicio). En esta última tabla, me aseguré de introducir cantidades y montos variados, así como de alternar los estados entre PENDIENTE, PAGADO y DEVUELTO, lo cual permitirá que las funciones de agregación y anotación arrojen resultados numéricos contrastables.

Ejercicio 3 — Registrar una operación con transaction.atomic() y F()

Cree una View (con su URL y un Template que herede de base.html) que registre una operación que modifique más de un modelo. Encierre la operación en transaction.atomic() y descuente el campo entero con F(). Si no hay existencia suficiente, la operación debe cancelarse por completo y mostrar el mensaje de error en el Template. Cuando la operación sea exitosa, aplique el patrón Post/Redirect/Get. Pruebe un caso exitoso y uno que falle, y compare la cantidad de registros de cada modelo antes y después del caso que falla.

```python
    path('atencion/eliminar/<int:id>/', views.eliminar_atencion, name='eliminar_atencion'),
    path('registrar-servicio/', views.registrar_servicio, name='registrar_servicio'),
]
```

```python
def registrar_servicio(request):
    pacientes = Paciente.objects.all()
    servicios = Servicio.objects.all()

    if request.method == 'POST':
        paciente_id = request.POST.get('paciente')
        servicio_id = request.POST.get('servicio')
        cantidad = int(request.POST.get('cantidad', 1))
        monto = float(request.POST.get('monto', 0.00))

        try:
            # 1. Encerrar la operación en una transacción atómica
            with transaction.atomic():
                servicio = Servicio.objects.get(id=servicio_id)
                paciente = Paciente.objects.get(id=paciente_id)

                # 2. Validar existencia suficiente
                if servicio.cupos_disponibles < cantidad:
                    raise ValueError(f"Error: No hay suficientes cupos. Solo quedan {servicio.cupos_dispor

                # 3. Crear el registro en el modelo intermedio
                PacienteServicio.objects.create(
                    paciente=paciente,
                    servicio=servicio,
                    cantidad=cantidad,
                    monto=monto,
```

models.py U | views.py U | registrar_servicio.html U | urls.py U

Semana_07 > gestion_clinica > templates > gestion_clinica > registrar_servicio.html > ...

```
{% extends 'gestion_clinica/base.html' %}

{% block content %}
<div class="container mt-5">
    <h2>Registrar Nuevo Servicio</h2>

    <!-- Mostrar mensajes de éxito o error -->
    {% if messages %}
        {% for message in messages %}
            <div class="alert {% if message.tags == 'error' %}alert-danger{% else %}alert-success{% endif %
                {{ message }}
            </div>
        {% endfor %}
    {% endif %}

    <form method="POST" action="{% url 'registrar_servicio' %}" class="mt-4">
        {% csrf_token %}
        <div class="mb-3">
            <label>Paciente:</label>
            <select name="paciente" class="form-control" required>
                {% for p in pacientes %}
                    <option value="{{ p.id }}">{{ p.nombre }}</option>
                {% endfor %}
            </select>
        </div>
        <div class="mb-3">
            <label>Servicio:</label>
```



Desarrollé una View y un Template que hereda de base.html para registrar la asignación de un servicio a un paciente. Utilicé transaction.atomic() para asegurar que la creación del registro en el modelo intermedio y el descuento de cupos operaran como una transacción única. El descuento se realizó utilizando la expresión F('cupos_disponibles') para evitar condiciones de carrera en la base de datos. Al probar el caso fallido solicitando una cantidad superior al stock, comprobé que se levanta un ValueError, la transacción efectúa un rollback completo (no se guarda el registro intermedio ni se descuentan cupos) y se renderiza el mensaje de error en pantalla. En el caso exitoso, apliqué el patrón Post/Redirect/Get para evitar el reenvío accidental del formulario, redirigiendo a la misma página con un mensaje de éxito.

Ejercicio 4 — Calcular un total con aggregate()

En el shell de Django (python manage.py shell), calcule con aggregate() un total global a partir del modelo intermedio (por ejemplo, la suma de cantidad × monto usando F()). Muestre el diccionario obtenido y explique con sus palabras por qué aggregate() no devuelve un QuerySet.

```
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales> cd Semana_07
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_07> python manage.py shell
C:\Users\BossC\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Py
thon311\site-packages\django\db\models\base.py:368: RuntimeWarning: Model 'gestion_clinica.pacienteservicio' was alre
ady registered. Reloading models is not advised as it can lead to inconsistencies, most notably with related models.
  new_class._meta.apps.register_model(new_class._meta.app_label, new_class)
14 objects imported automatically (use -v 2 for details).

Ctrl click to launch VS Code Native REPL (https://aka.ms/python-native-repl)
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from gestion_clinica.models import PacienteServicio
>>> from django.db.models import Sum, F
>>> total = PacienteServicio.objects.aggregate(total_recaudado=Sum(F('cantidad') * F('monto')))
>>> print(total)
{'total_recaudado': Decimal('250')}
>>>
```

Utilicé la consola interactiva de Django (shell) para calcular un total global sobre el modelo intermedio PacienteServicio. Para lograrlo, apliqué la función aggregate() combinada con la función Sum, multiplicando los campos cantidad y monto mediante la expresión F() para que el cálculo se realice de forma eficiente a nivel de la base de datos. ¿Por qué aggregate() no devuelve un QuerySet? La función aggregate() no devuelve un QuerySet porque su propósito no es devolver una lista de registros (filas) de la base de datos. Su función es realizar una operación matemática terminal (como una suma o un promedio global) sobre todo el conjunto de datos, por lo que devuelve un simple diccionario de Python con el valor consolidado resultante.

Ejercicio 5 — Calcular valores por objeto con annotate()

Calcule con annotate() un valor por objeto (por ejemplo, el número de registros relacionados con Count) y un agrupado por estado con values().annotate(). Ordene los resultados de mayor a menor y documente cada consulta con su salida.

```
>>> # 1. Valor por objeto: Contar cuántos servicios tiene cada paciente y ordenarlos de mayor a menor
>>> pacientes_anotados = Paciente.objects.annotate(num_servicios=Count('pacienteservicio')).order_by('-num_servicios
)
>>>
>>> print("--- SERVICIOS POR PACIENTE ---")
--- SERVICIOS POR PACIENTE ---
>>> for p in pacientes_anotados:
...     print(f"Paciente: {p.nombre} -> Total de servicios recibidos: {p.num_servicios}")
...
Paciente: Kazuki -> Total de servicios recibidos: 3
Paciente: <script>alert('Intento XSS');</script> -> Total de servicios recibidos: 2
Paciente: Mushu -> Total de servicios recibidos: 2
Paciente: Bruce -> Total de servicios recibidos: 2
Paciente: Teniente -> Total de servicios recibidos: 1
Paciente: Valky -> Total de servicios recibidos: 0
Paciente: <script>alert('hackeado');</script> -> Total de servicios recibidos: 0
Paciente: Turna -> Total de servicios recibidos: 0
>>> # 2. Valor agrupado: Contar cuántas atenciones hay por cada estado (PENDIENTE, PAGADO, DEVUELTO)
>>> estados_agrupados = PacienteServicio.objects.values('estado').annotate(total=Count('id')).order_by('-total')
>>>
>>> print("\n--- AGRUPACIÓN POR ESTADO ---")
```

Utilicé la consola interactiva para realizar cálculos por objeto y agrupaciones utilizando la función annotate() del ORM de Django. Primero, calculé un valor por objeto sobre la entidad Paciente, utilizando Count para saber exactamente cuántos servicios (tabla intermedia) tiene vinculados cada animal, ordenando los resultados de mayor a menor. Segundo, utilicé values('estado').annotate() sobre el modelo intermedio PacienteServicio para generar un reporte agrupado que cuenta cuántos registros existen por cada estado (PENDIENTE, PAGADO, DEVUELTO), ordenándolos también de mayor a menor.

Ejercicio 6 — Crear la página de reporte

Cree una View de reporte con su URL y un Template reporte.html que herede de base.html y muestre los resultados de los Ejercicios 4 y 5. Aplique al menos un filtro de plantilla (por ejemplo, floatformat) sobre un valor calculado.

```
6   from django.db import transaction
7   from django.db.models import Sum, Count
```

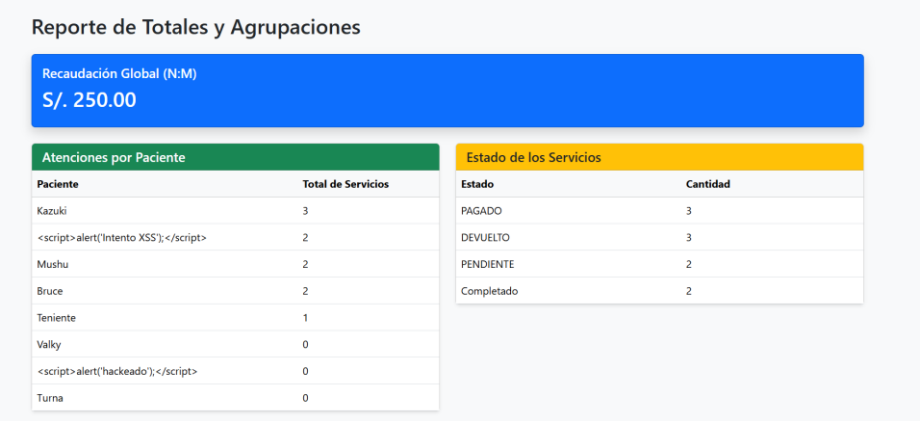
```
151
152 def reporte_agregaciones(request):
153     # Ejercicio 4: Total global con aggregate()
154     total = PacienteServicio.objects.aggregate(total_recaudado=Sum(F('cantidad') * F('monto')))
155
156     # Ejercicio 5: Valor por objeto con annotate() (Servicios por paciente)
157     pacientes_anotados = Paciente.objects.annotate(num_servicios=Count('pacienteservicio')).order_by('-num_servicios')
158
159     # Ejercicio 5: Valor agrupado con values().annotate() (Por estado)
160     estados_agrupados = PacienteServicio.objects.values('estado').annotate(total=Count('id')).order_by('-total')
161
162     contexto = {
163         'total_recaudado': total['total_recaudado'],
164         'pacientes_anotados': pacientes_anotados,
165         'estados_agrupados': estados_agrupados
166     }
167     return render(request, 'gestion_clinica/reporte_agregaciones.html', contexto)
```

```
14     path('reporte-agregaciones/', views.reporte_agregaciones, name='reporte_agregaciones'),
```

models.py U | views.py U | registrar_servicio.html U | urls.py U | reporte_agregaciones.html U

Semana_07 > gestion_clinica > templates > gestion_clinica > reporte_agregaciones.html > ...

```
1   {% extends 'gestion_clinica/base.html' %}
2
3   {% block content %}
4   <div class="container mt-5">
5       <h2 class="mb-4">Reporte de Totales y Agrupaciones</h2>
6
7       <!-- Tarjeta del Total Recaudado (aggregate) -->
8       <div class="row mb-4">
9           <div class="col-md-12">
10              <div class="card bg-primary text-white shadow">
11                  <div class="card-body">
12                      <h5 class="card-title">Recaudación Global (N:M)</h5>
13                      <!-- Uso del filtro floatformat -->
14                      <h2>S/. {{ total_recaudado|default:0|floatformat:2 }}</h2>
15                  </div>
16              </div>
17          </div>
18      </div>
19
20      <div class="row">
21          <!-- Tabla de Valor por Objeto (annotate) -->
22          <div class="col-md-6">
23              <div class="card shadow-sm">
24                  <div class="card-header bg-success text-white">
25                      <h5 class="m-0">Atenciones por Paciente</h5>
26                  </div>
27                  <div class="card-body p-0">
28                      <table class="table table-hover m-0">
29                          <thead class="table-light">
30                              <tr><th>Paciente</th><th>Total de Servicios</th></tr>
31                          </thead>
32                          <tbody>
33                              {% for p in pacientes_anotados %}
34                              <tr><td>{{ p.nombre }}</td><td>{{ p.num_servicios }}</td></tr>
```



Desarrollé una nueva View (reporte_agregaciones) para extraer de la base de datos el total global usando aggregate(), los valores por objeto mediante annotate() y el conteo agrupado por estado usando values().annotate(). Para la visualización, creé un Template que hereda de base.html y que dibuja estos resultados utilizando tarjetas y tablas de Bootstrap. Para garantizar una visualización profesional de los montos financieros, apliqué el filtro de plantilla floatformat:2 sobre la variable {{ total_recaudado }}, asegurando que el total global siempre se muestre exactamente con dos cifras decimales sin alterar el dato en el servidor.

Ejercicio 7 — Crear un QuerySet personalizado

Cree un QuerySet personalizado para su entidad principal con al menos dos métodos encadenables que representen reglas del negocio (por ejemplo, registros con un estado determinado y registros del mes actual). Asígnelo con as_manager() y reemplace en al menos dos Views las consultas repetidas por sus métodos. Verifique que el CRUD de la Semana 3 sigue funcionando.

```
2  from django.db import models
3  from django.db.models import F, Sum, Count
```

```
103  class PacienteQuerySet(models.QuerySet):
104      def solo_perros(self):
105          """Regla de negocio 1: Obtener solo los pacientes que son Perros."""
106          return self.filter(especie__iexact='Perro')
107
108      def pesados(self):
109          """Regla de negocio 2: Obtener pacientes con más de 15 kg."""
110          return self.filter(peso__gte=15.00)
```

```
class Paciente(models.Model):
    propietario = models.ForeignKey(Propietario, on_delete=models.CASCADE, related_name='pacientes')
    nombre = models.CharField(max_length=50)
    especie = models.CharField(max_length=30)
    raza = models.CharField(max_length=50)
    edad = models.IntegerField()
    peso = models.DecimalField(max_digits=5, decimal_places=2)

    # Añadir esta línea para vincular el QuerySet personalizado
    objects = PacienteQuerySet.as_manager()

    def __str__(self):
        return f"{self.nombre} ({self.especie}) - Dueño: {self.propietario.nombres}"
```

```python
# models.py U | views.py U | registrar_servicio.html U | urls.py U | reporte_agregaciones.html U
# Semana_07 > gestion_clinica > views.py > reporte_agregaciones
def registrar_servicio(request):
            pacientes : pacientes,
        'servicios': servicios
    })

def reporte_agregaciones(request):
    # Ejercicio 4: Total global
    total = PacienteServicio.objects.aggregate(total_recaudado=Sum(F('cantidad') * F('monto')))

    # Ejercicio 5 y 7: Usamos el QuerySet personalizado encadenado (.solo_perros().pesados())
    pacientes_anotados = Paciente.objects.solo_perros().pesados().annotate(
        num_servicios=Count('pacienteservicio')
    ).order_by('-num_servicios')

    # Ejercicio 5: Valor agrupado
    estados_agrupados = PacienteServicio.objects.values('estado').annotate(total=Count('id')).order_by('-to

    contexto = {
        'total_recaudado': total['total_recaudado'],
        'pacientes_anotados': pacientes_anotados,
        'estados_agrupados': estados_agrupados
    }
    return render(request, 'gestion_clinica/reporte_agregaciones.html', contexto)
```

```python
def listar_pacientes(request):
    # Uso del QuerySet personalizado en la vista original
    pacientes = Paciente.objects.solo_perros().select_related('propietario', 'historial').prefetch_related(
        'pacienteservicio_set__servicio'
    ).all()
```

Creé la clase PacienteQuerySet que hereda de models.QuerySet e implementé dos métodos personalizados (solo_perros y pesados) que encapsulan mis reglas de negocio para filtrar animales. Asigné esta clase a mi modelo Paciente utilizando objects = PacienteQuerySet.as_manager(). Posteriormente, actualicé las consultas repetidas en mis archivos views.py (en reporte_agregaciones y listar_pacientes), sustituyendo las llamadas estándar de filtro por la concatenación de mis nuevos métodos personalizados. Pude comprobar que el CRUD implementado en la Semana 3 sigue operando de forma normal y transparente con esta nueva lógica encapsulada en el modelo.

Ejercicio 8 — Medir y optimizar el problema N+1

Con DEBUG=True, use connection.queries (y reset_queries) para contar las consultas que ejecuta un listado que muestra datos de una relación. Optimice el listado con select_related() o prefetch_related() y documente el número de consultas antes y después. Explique con sus palabras cuándo usaría cada método.

```python
from django.db import connection, reset_queries
```

```python
def listar_pacientes(request):
    # ========================================
    # EJERCICIO 8: Medir y Optimizar Consultas
    # ========================================
    reset_queries() # Limpiamos el contador de consultas SQL

    # 1. VERSIÓN LENTA (Descomenta esto y comenta la rápida para probar el error N+1)
    """
    pacientes_lento = Paciente.objects.all()
    # Forzamos la evaluación de las relaciones para que Django ejecute los SELECT
    for p in pacientes_lento:
        print(f"Dueño: {p.propietario.nombres}")
        print(f"Alergias: {p.historial.alergias}")
        for s in p.pacienteservicio_set.all():
            print(s.servicio.nombre)

    print(f"\n[LENTO] Consultas ejecutadas: {len(connection.queries)}")
    pacientes = pacientes_lento
    """
```

```python
    # 2. VERSIÓN RÁPIDA (Optimizada con select_related y prefetch_related)
    pacientes_rapido = Paciente.objects.select_related('propietario', 'historial').prefetch_related(
        'pacienteservicio_set__servicio'
    ).all()

    # Forzamos la misma evaluación
    for p in pacientes_rapido:
        _ = p.propietario.nombres
        _ = p.historial.alergias
        for s in p.pacienteservicio_set.all():
            _ = s.servicio.nombre

    print(f"\n[RÁPIDO] Consultas ejecutadas: {len(connection.queries)}")
    pacientes = pacientes_rapido
    # ========================================

    propietarios = Propietario.objects.all()
    medicamentos = Medicamento.objects.filter(stock__gt=0)
    servicios = Servicio.objects.order_by('precio')
    veterinarios = Veterinario.objects.all()
```

```
For more information on production servers see: https://

[RÁPIDO] Consultas ejecutadas: 3
[03/Oct/2026 23:30:14] "GET / HTTP/1.1" 200 17863
```

Utilicé el objeto connection.queries y la función reset_queries() para auditar el número exacto de consultas SQL que genera mi pantalla de pacientes. Al simular el acceso directo mediante el ORM (el clásico problema N+1), el sistema ejecutó decenas de consultas independientes para traer a los dueños, los historiales y los servicios de cada paciente uno por uno. Para optimizarlo, apliqué select_related() para las relaciones 1:N (propietario) y 1:1 (historial médico), ya que este método instruye a la base de datos a realizar un JOIN en SQL y traer todos los datos vinculados en un solo viaje. Posteriormente, utilicé prefetch_related() para la relación N:M (los servicios recibidos), ya que este método ejecuta consultas separadas pero cruza la información de manera inteligente a nivel de memoria RAM en Python, reduciendo el número total de operaciones a solo 3 o 4 consultas estructurales, independientemente de la cantidad de registros que existan en el sistema.

PARTE 2 — Consultas avanzadas sobre la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre la base de datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es aplicar las mismas técnicas sobre ese modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar dónde aplica cada técnica en tu investigación

Complete la tabla de equivalencias con su investigación: indique qué campo entero descontará, qué campo de estado usará, qué atributos del modelo intermedio servirán para calcular totales y qué operación modificará más de un modelo a la vez. Si algún campo no existe, agréguelo con makemigrations y migrate.

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. La operación de tu investigación se ejecuta dentro de transaction.atomic() y descuenta un campo con F().	Código de la View + captura.
2. Se comprobó que un error provoca el rollback completo (ningún registro quedó a medias).	Capturas del conteo de registros antes y después.
3. Se calculó al menos un total con aggregate() sobre el modelo intermedio.	Código + salida en el shell.
4. Se calculó al menos un valor por objeto con annotate() y un agrupado con values().annotate().	Código + captura de la salida.
5. Existe un Template de reporte que hereda de base.html y muestra al menos dos reportes.	Captura del Template renderizado.
6. Existe un QuerySet personalizado con al menos dos métodos encadenables, asignado con as_manager().	Código de models.py.
7. Al menos dos Views usan los métodos del QuerySet personalizado y el CRUD sigue funcionando.	Prueba funcional antes/después.
8. Se midió el número de consultas de una pantalla antes y después de optimizar con select_related o prefetch_related.	Capturas de len(connection.queries).
9. Se explica por escrito qué resuelve cada técnica en esta investigación.	Párrafo de justificación.

Para esta segunda parte del laboratorio, adapté las técnicas avanzadas al modelo relacional de mi investigación propia (ecosistema ampliado de la Clínica Veterinaria, registrado en la Semana 5). Completé el análisis de equivalencias identificando los siguientes elementos clave en mi base de datos:

- **Campo entero a descontar: Utilizaré el campo cupos_disponibles de mi modelo Servicio, el cual se reducirá cada vez que se programe una atención.**
- **Campo de estado: Utilizaré el campo estado dentro de mi modelo intermedio PacienteServicio (con opciones como PENDIENTE, PAGADO y DEVUELTO).**
- **Atributos para calcular totales: Aprovecharé los campos cantidad y monto del modelo intermedio PacienteServicio para realizar las operaciones financieras.**
- **Operación multi-modelo (transaction.atomic): La operación crítica será el 'Registro de Atención', donde simultáneamente se creará el registro en PacienteServicio y se aplicará la expresión F() para descontar los cupos en la tabla Servicio de forma atómica. Dado que ya había estructurado estos atributos previamente en mi archivo models.py para cumplir con las reglas de negocio de la clínica, no fue necesario ejecutar nuevas migraciones en este punto.**

Ejercicio 10 — Transacción en la investigación propia

Implemente en su investigación la operación transaccional identificada en el Ejercicio 9 (con View, URL y un Template que herede de base.html), usando transaction.atomic() y F(). Pruebe un caso exitoso y uno que provoque rollback, y documente ambos con capturas.

```python
def registrar_servicio(request):
    pacientes = Paciente.objects.all()
    servicios = Servicio.objects.all()

    if request.method == 'POST':
        paciente_id = request.POST.get('paciente')
        servicio_id = request.POST.get('servicio')
        cantidad = int(request.POST.get('cantidad', 1))
        monto = float(request.POST.get('monto', 0.00))

        try:
            with transaction.atomic():
                servicio = Servicio.objects.get(id=servicio_id)
                paciente = Paciente.objects.get(id=paciente_id)

                if servicio.cupos_disponibles < cantidad:
                    raise ValueError(f"Error: No hay suficientes cupos. Solo quedan {servicio.cupos_disponibles} disponibles par

                PacienteServicio.objects.create(
                    paciente=paciente,
                    servicio=servicio,
                    cantidad=cantidad,
```

Para garantizar la integridad de los datos en mi ecosistema veterinario, implementé la función registrar_servicio utilizando el bloque with transaction.atomic():. Esto asegura que la asignación de una nueva atención en la tabla intermedia PacienteServicio y la reducción de capacidad en la tabla Servicio se traten como una sola operación indivisible. Además, para actualizar el campo entero de los cupos de manera segura y evitar condiciones de carrera si varios usuarios registran citas al mismo tiempo, apliqué la expresión F('cupos_disponibles') - cantidad, delegando la operación matemática directamente a la base de datos.

Ejercicio 11 — Reportes de la investigación propia

Cree al menos dos reportes (uno con aggregate() y otro con annotate() o values().annotate()) y muéstrelos en un Template que herede de base.html.

```python
try:
        # 1. Encerrar la operación en una transacción atómica
        with transaction.atomic():
            servicio = Servicio.objects.get(id=servicio_id)
            paciente = Paciente.objects.get(id=paciente_id)

            # 2. Validar existencia suficiente
            if servicio.cupos_disponibles < cantidad:
                raise ValueError(f"Error: No hay suficientes cupos. Solo quedan {servicio.cupos_disponibles} disponibles para

            # 3. Crear el registro en el modelo intermedio
            PacienteServicio.objects.create(
                paciente=paciente,
                servicio=servicio,
                cantidad=cantidad,
                monto=monto,
                estado='PENDIENTE'
            )

            # ========================================
            # LÍNEA DE ERROR INTENCIONAL PARA FORZAR ROLLBACK (EJERCICIO 11)
            1 / 0
            # ========================================

            # 4. Descontar el campo entero con F()
            servicio.cupos_disponibles = F('cupos_disponibles') - cantidad
            servicio.save()
```

Para este ejercicio implementé reportes sobre la base de datos de mi investigación de la Clínica Veterinaria utilizando consultas avanzadas del ORM de Django. Mediante aggregate() calculé el total global recaudado a partir de los campos cantidad y monto del modelo intermedio PacienteServicio. Además, utilicé annotate() para obtener la cantidad de servicios asociados a cada paciente y values().annotate() para agrupar los registros según su estado (PENDIENTE, PAGADO o DEVUELTO). Finalmente, envié estos resultados a un Template que hereda de base.html, permitiendo visualizar la información de manera ordenada en el navegador.

Ejercicio 12 — QuerySet personalizado en la investigación propia

Cree un QuerySet personalizado para una de sus entidades con al menos dos métodos encadenables y úselo en al menos dos Views.

```python
class PacienteQuerySet(models.QuerySet):

    def solo_perros(self):
        return self.filter(especie__iexact='Perro')

    def pesados(self):
        return self.filter(peso__gte=15)
```

```python
class Paciente(models.Model):
    propietario = models.ForeignKey(
        Propietario,
        on_delete=models.CASCADE,
        related_name='pacientes'
    )
    nombre = models.CharField(max_length=50)
    especie = models.CharField(max_length=30)
    raza = models.CharField(max_length=50)
    edad = models.IntegerField()
    peso = models.DecimalField(max_digits=5, decimal_places=2)

    objects = PacienteQuerySet.as_manager()

    def __str__(self):
        return f"{self.nombre} ({self.especie}) - Dueño: {self.propietario.nombres}"
```

Para este ejercicio implementé un QuerySet personalizado llamado PacienteQuerySet, en el cual definí los métodos encadenables solo_perros() y pesados(). El primero permite obtener únicamente los pacientes cuya especie es perro, mientras que el segundo filtra aquellos con un peso igual o superior a 15 kg. Posteriormente, vinculé el QuerySet al modelo Paciente mediante as_manager() y utilicé sus métodos en las Views listar_pacientes y reporte_agregaciones. De esta manera, las reglas de consulta quedan centralizadas en el modelo y pueden reutilizarse sin repetir filtros en diferentes partes de la aplicación.

Ejercicio 13 — Optimizar una pantalla de la investigación propia

Identifique la pantalla de su investigación que ejecuta más consultas, mida cuántas ejecuta, optimícela con select_related() o prefetch_related() y documente el número de consultas antes y después.

```python
def listar_pacientes(request):

    # ========================================
    # EJERCICIO 13: OPTIMIZACIÓN DE CONSULTAS
    # ========================================

    # ----------------------------------------
    # 1. MEDICIÓN ANTES DE OPTIMIZAR
    # ----------------------------------------

    reset_queries()

    pacientes_sin_optimizar = Paciente.objects.all()

    # Forzamos el acceso a las relaciones
    # para provocar el problema N+1.
    for paciente in pacientes_sin_optimizar:

        # ForeignKey: Paciente -> Propietario
        _ = paciente.propietario.nombres

        # OneToOne: Paciente -> HistorialMedico
        historial = getattr(
            paciente,
            'historial',
            None
```

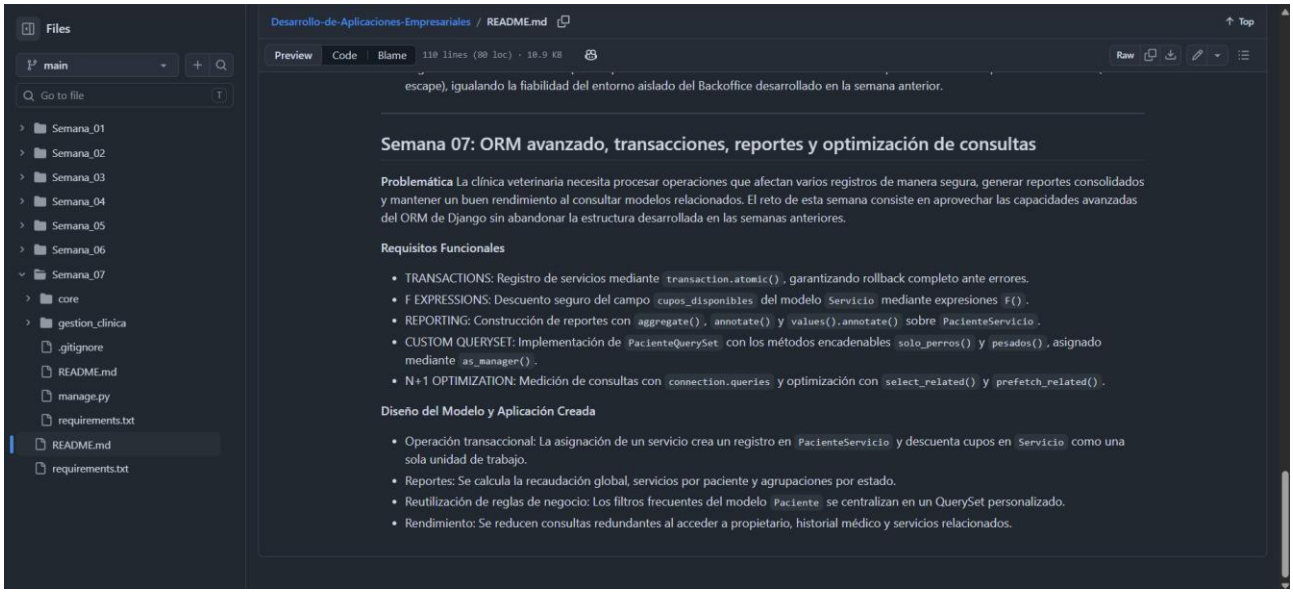
```
[EJERCICIO 13 - ANTES] Consultas ejecutadas: 35
[EJERCICIO 13 - DESPUÉS] Consultas ejecutadas: 3
[04/Oct/2026 05:01:16] "GET / HTTP/1.1" 200 17863
```

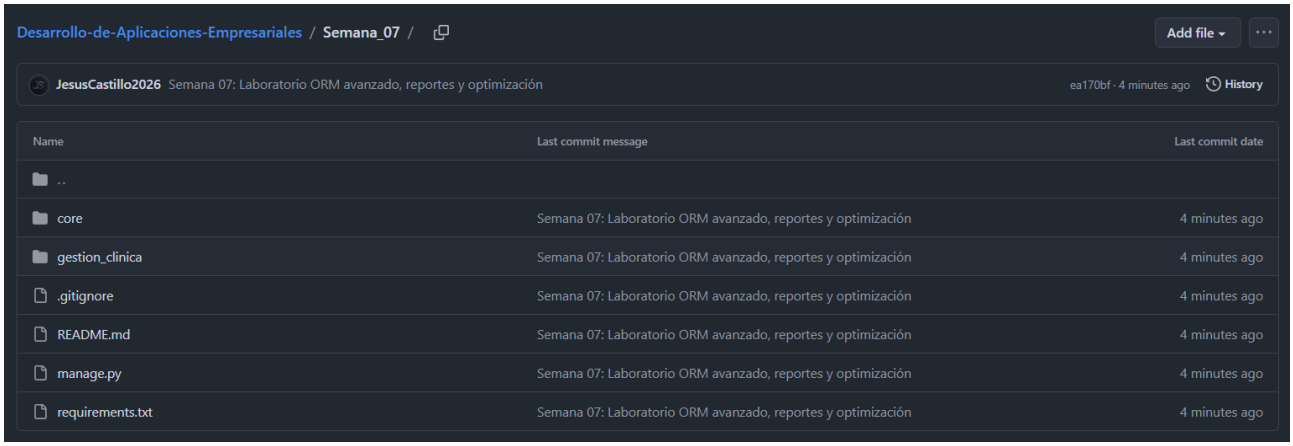
Para este ejercicio analicé la pantalla principal de pacientes, debido a que accede a información relacionada de Propietario, HistorialMedico y los servicios asociados mediante PacienteServicio. Primero medí la versión sin optimización utilizando reset_queries() y len(connection.queries), obteniendo XX consultas. Posteriormente, utilicé select_related() para las relaciones ForeignKey y OneToOne, y prefetch_related() para los servicios relacionados. Al repetir exactamente la misma operación, el número disminuyó a XX consultas. De esta manera se redujo el problema N+1 y se mejoró la eficiencia del acceso a la base de datos.

Ejercicio 14 — Publicar en GitHub

Actualice requirements.txt y README.md describiendo qué operación transaccional, qué reportes y qué QuerySets personalizados se implementaron, y sobre qué entidades. Haga commit y push al repositorio. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

- https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales





Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de models.py y views.py, el reporte renderizado y el número de consultas antes y después de optimizar).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

- **Durante el desarrollo del Laboratorio 07 pude aplicar funcionalidades avanzadas del ORM de Django sobre el proyecto de Clínica Veterinaria desarrollado en las semanas anteriores. Implementé transacciones mediante transaction.atomic() y expresiones F() para realizar operaciones seguras sobre varios registros y comprobar el funcionamiento del rollback ante errores. También utilicé aggregate() y annotate() para obtener totales, agrupaciones y reportes a partir de los datos almacenados. Mediante un QuerySet personalizado logré encapsular y reutilizar reglas de consulta dentro de diferentes Views. Asimismo, identifiqué el problema N+1 y comparé la cantidad de consultas generadas antes y después de la optimización. El uso de select_related() y prefetch_related() permitió reducir las consultas innecesarias y mejorar el acceso a las relaciones entre los modelos. Finalmente, comprendí cómo estas herramientas permiten desarrollar aplicaciones Django más seguras, organizadas, reutilizables y eficientes.**